

VulScan

© Marco Lungo & Claude Mirrors

Valutazione di sicurezza

Sistema di voto telematico — analisi statica del codice sorgente

Repository: votazioni-main

Oggetto	Applicazione Django di gestione delle votazioni
Tipo di attività	Revisione statica del codice (secure code review)
Data	14 luglio 2026
Rilievi totali	18 (12 certi, 6 possibili/condizionati)
Severità critica/alta	8 rilievi
Classificazione	Riservato

Le valutazioni di severità e probabilità costituiscono un giudizio professionale e dipendono dal contesto di installazione.

Analisi tecnica condotta su base volontaria, su richiesta di un socio dell'Associazione, sulla base del solo codice sorgente.

Indice

Indice	2
1. Sintesi esecutiva.....	3
2. Ambito, metodologia e limiti	3
3. Quadro dei risultati	3
4. Dettaglio delle vulnerabilità.....	4
V1 — Protezione CSRF disabilitata globalmente	4
V2 — Credenziali email in chiaro nel codice sorgente.....	5
V3 — SECRET_KEY hardcoded e di tipo "django-insecure"	5
V4 — Nessuna validazione della scheda: candidato/categoria arbitrari	6
V5 — Token di voto di tutti gli elettori in copia a una casella condivisa	6
V6 — Segretezza del voto vanificabile per correlazione temporale.....	7
V8 — Django 3.2.7 in End-of-Life con CVE note non corrette	7
V9 — Cookie di sessione e trasporto privi di direttive di sicurezza	8
V7 — Ricevuta verificabile: rischio di coercizione e voto di scambio	8
V10 — Limite ai campi POST disattivato: proliferazione record e DoS.....	9
V11 — Password amministratore distribuita via email e BCC	9
V13 — Helper di servizio file statici di Django in produzione	10
V14 — Login amministrativo privo di rate limiting e CAPTCHA.....	10
V15 — SQLite come database di produzione	10
V17 — conf.json e template email modificabili via web	11
V18 — Gestione errori disabilitata e DEBUG attivabile: rischio info leak.....	11
V12 — Token di voto trasmesso come parametro nell'URL	12
V16 — Invio email bloccante nel contesto della richiesta	12
5. Correzioni già presenti nel codice.....	13
6. Raccomandazioni prioritzate.....	13
Immediata (0–7 giorni)	13
Breve termine (2–4 settimane).....	13
Medio termine (1–3 mesi)	13
7. Considerazioni di competenza e protezione dei dati.....	14
8. Appendici	14
A. Riferimenti	14
B. Glossario essenziale	14
C. Avvertenza	14

1. Sintesi esecutiva

L'analisi del codice sorgente dell'applicazione di voto ha individuato **18 rilievi di sicurezza: 12 vulnerabilità certe** (difetti verificabili direttamente nel codice) e **6 possibili o condizionate** al contesto di installazione. Otto rilievi hanno severità critica o alta.

Il quadro complessivo indica un'applicazione tuttora acerba sul piano della sicurezza, con criticità che toccano i due valori fondanti di un sistema elettorale — l'integrità e la segretezza del voto — oltre a errori di configurazione e di gestione dei segreti. Alcuni interventi correttivi risultano già presenti nel codice (cfr. §5), segno di una manutenzione in corso ma non completata.

Priorità immediate (da affrontare prima di qualunque nuovo utilizzo in produzione):

- Revocare e rigenerare le credenziali email e la password amministratore esposte nel codice (V2, V11) e sostituire la SECRET_KEY (V3).
- Riabilitare la protezione anti-CSRF, disattivata a livello globale (V1).
- Introdurre la validazione lato server delle preferenze di voto e reintrodurre il limite ai campi della richiesta (V4, V10).
- Eliminare l'invio in copia dei token individuali di voto verso la casella condivisa (V5).

Nota di merito: la protezione contro il doppio voto e la rimozione del backdoor storico "MENZA_SECRET" risultano già implementate. Il lavoro di messa in sicurezza è quindi avviato ma va completato, con particolare attenzione allo schema di urna che oggi rende la segretezza del voto solo apparente (V6).

2. Ambito, metodologia e limiti

Ambito: Revisione statica del codice sorgente contenuto nel repository votazioni-main (applicazione Django, Python), comprensivo di configurazione, viste, modelli, template, comandi di gestione e file di build.

Metodologia: Lettura manuale del codice e della configurazione, con classificazione dei rilievi per severità e probabilità e mappatura alle categorie OWASP Top 10 (2021) e CWE. Le versioni delle dipendenze sono state verificate rispetto alle CVE note e allo stato di supporto (End-of-Life).

Fuori ambito: Non sono stati eseguiti test dinamici o penetration test sull'applicazione in esecuzione, né analisi dell'infrastruttura di produzione (rete, TLS, sistema operativo, posta), né revisione delle dipendenze di terze parti riga per riga.

Limiti e avvertenze: Le stime di severità e probabilità sono giudizi professionali e dipendono dal modello di minaccia e dal contesto di installazione (esposizione dell'host, presenza di reverse proxy/HTTPS, titolarità delle caselle email). L'assenza di rilievi in un'area non implica assenza di vulnerabilità.

Scala di severità: Critica (compromissione diretta di integrità/segretezza del voto o dei segreti) · Alta (impatto rilevante su sicurezza o processo elettorale) · Media (impatto significativo ma condizionato) · Bassa (impatto limitato o fortemente mitigato).

3. Quadro dei risultati

La tabella seguente riepiloga i 18 rilievi. Il dettaglio è nel §4. Una rappresentazione a matrice (severità × probabilità) accompagna il presente documento nella vista interattiva.

ID	Vulnerabilità	Severità	Prob.	Stato
V1	Protezione CSRF disabilitata globalmente	Critica	Alta	Certa

ID	Vulnerabilità	Severità	Prob.	Stato
V2	Credenziali email in chiaro nel codice sorgente	Critica	Alta	Certa
V3	SECRET_KEY hardcoded e di tipo "django-insecure"	Alta	Media	Certa
V4	Nessuna validazione della scheda: candidato/categoria arbitrari	Alta	Alta	Certa
V5	Token di voto di tutti gli elettori in copia a una casella condivisa	Alta	Media	Certa
V6	Segretezza del voto vanificabile per correlazione temporale	Alta	Media	Certa
V7	Ricevuta verificabile: rischio di coercizione e voto di scambio	Media	Media	Certa
V8	Django 3.2.7 in End-of-Life con CVE note non corrette	Alta	Media	Certa
V9	Cookie di sessione e trasporto privi di direttive di sicurezza	Alta	Media	Certa
V10	Limite ai campi POST disattivato: proliferazione record e DoS	Media	Media	Certa
V11	Password amministratore distribuita via email e BCC	Media	Media	Certa
V12	Token di voto trasmesso come parametro nell'URL	Bassa	Media	Certa
V13	Helper di servizio file statici di Django in produzione	Media	Bassa	Possibile
V14	Login amministrativo privo di rate limiting e CAPTCHA	Media	Bassa	Possibile
V15	SQLite come database di produzione	Media	Media	Possibile
V16	Invio email bloccante nel contesto della richiesta	Bassa	Media	Possibile
V17	conf.json e template email modificabili via web	Media	Bassa	Possibile
V18	Gestione errori disabilitata e DEBUG attivabile: rischio info leak	Media	Bassa	Possibile

4. Dettaglio delle vulnerabilità

V1 — Protezione CSRF disabilitata globalmente

Severità	Critica
Probabilità	Alta
Stato	Certa
Categoria	OWASP A01:2021 – Broken Access Control · CWE-352
Posizione	mensa_italia_voting/settings.py:54

Descrizione: Il middleware `django.middleware.csrf.CsrfViewMiddleware` è commentato nella catena `MIDDLEWARE`: la protezione anti-CSRF di Django è quindi disattivata a livello globale, nonostante i template continuino a emettere `{% csrf_token %}` (token generato ma mai verificato). La view di voto ignora inoltre esplicitamente i campi che contengono la stringa `"csrf"`.

Impatto: Ogni richiesta POST è falsificabile da un sito terzo mentre la vittima è autenticata. Sono esposte in particolare le funzioni amministrative di stato: scrittura di file sul server (`conf.json` e template email tramite `/management/edit`), caricamento dell'elenco aventi diritto (`/management/upload`), invio massivo delle credenziali (`/management/send`) e la stessa login dell'area `/admin`.

Scenario: Un amministratore autenticato che apra una pagina malevola può, a sua insaputa, veder sovrascritto `conf.json` (candidati, finestra temporale, modalità di voto) o innescato l'invio massivo di email ai soci.

Raccomandazione: Riabilitare `CsrfViewMiddleware`; mantenere `{% csrf_token %}` in tutti i form e verificarlo lato server; rimuovere la logica che scarta i campi `"csrf"`; non utilizzare `@csrf_exempt`.

V2 — Credenziali email in chiaro nel codice sorgente

Severità	Critica
Probabilità	Alta
Stato	Certa
Categoria	OWASP A05:2021 – Security Misconfiguration · CWE-798
Posizione	<code>mensa_italia_voting/settings.py:141, 146-150</code>

Descrizione: In `settings.py` sono presenti in chiaro la password SMTP dell'account Gmail `amministratore@mensa.it` (`EMAIL_HOST_PASSWORD`) e, in un blocco commentato, una precedente API key del servizio MailerSend. Il file è versionato nel repository.

Impatto: Chiunque abbia accesso al codice (repository, backup, copie locali) ottiene credenziali valide per inviare posta a nome di Mensa Italia, con possibilità di phishing verso i soci e danno reputazionale. La chiave MailerSend, ancorché commentata, resta un segreto esposto finché non revocata.

Scenario: Fuoriuscita del repository o di una copia locale → uso della casella istituzionale per inviare comunicazioni fraudolente agli aventi diritto.

Raccomandazione: Revocare e rigenerare IMMEDIATAMENTE entrambe le credenziali. Spostare i segreti in variabili d'ambiente o in un secret manager, mai nel codice. Riscrivere la cronologia Git se il repository è condiviso. Rimuovere del tutto il blocco commentato.

V3 — SECRET_KEY hardcoded e di tipo "django-insecure"

Severità	Alta
Probabilità	Media
Stato	Certa
Categoria	OWASP A02:2021 – Cryptographic Failures · CWE-321/CWE-798
Posizione	<code>mensa_italia_voting/settings.py:23</code>

Descrizione: SECRET_KEY è un valore statico del tipo "django-insecure-...", generato dallo scaffolding di sviluppo e committato in chiaro.

Impatto: La chiave firma token e valori crittografici di Django (token di reset password, valori di django.core.signing, cookie dei messaggi). Una chiave nota/prevedibile riduce le garanzie di integrità di questi meccanismi. Con sessioni su database (default) il cookie di sessione non è direttamente falsificabile dalla sola SECRET_KEY, ma l'esposizione resta un difetto grave, aggravato dall'eventuale condivisione della chiave tra ambienti.

Scenario: Riutilizzo della chiave nota per forgiare valori firmati o token, ove i relativi flussi siano attivi.

Raccomandazione: Generare una SECRET_KEY casuale robusta, univoca per ciascun ambiente, letta da variabile d'ambiente; invalidare quella attuale; forzare la rigenerazione delle sessioni dopo la sostituzione.

V4 — Nessuna validazione della scheda: candidato/categoria arbitrari

Severità	Alta
Probabilità	Alta
Stato	Certa
Categoria	OWASP A04:2021 – Insecure Design · CWE-20
Posizione	voting/views.py:242-250 · voting/templates/vote/matrix.html

Descrizione: Nella gestione POST della view vote(), il server itera su tutti i campi del form e costruisce i record Vote spezzando il valore ricevuto (formato "candidato:categoria") senza alcuna validazione rispetto ai candidati e alle opzioni definiti in conf.json. Il valore è interamente controllato dal client (radio value="{{candidate.id}}:{{option}}").

Impatto: Un elettore legittimo (in possesso di un token valido) può inviare preferenze per candidati o categorie arbitrari o inesistenti e, non essendovi limite al numero di campi (cfr. V10), generare un numero elevato di record, alterando i conteggi. Il vincolo unique_together (voteKey, category, candidate) non protegge, perché voteKey è un UUID nuovo per ogni scheda: variando la stringa "categoria" si creano righe multiple per lo stesso candidato.

Scenario: Manipolazione dell'esito da parte di un singolo avente diritto, tramite richiesta POST costruita ad hoc.

Raccomandazione: Validare lato server ogni preferenza contro l'elenco chiuso di candidati e opzioni di conf.json; rifiutare l'intera scheda in presenza di un valore non previsto; imporre il numero atteso di preferenze per categoria; ripristinare un limite ai campi POST (cfr. V10).

V5 — Token di voto di tutti gli elettori in copia a una casella condivisa

Severità	Alta
Probabilità	Media
Stato	Certa
Categoria	OWASP A02/A01:2021 · CWE-522 · Privacy
Posizione	voting/management/commands/send_all_mail.py:35 · voting/views.py:257

Descrizione: Il comando di invio massivo inserisce in copia nascosta (BCC) l'indirizzo `votazionimensaitalia@gmail.com` in ogni email di credenziali, che contiene il token individuale di voto dell'elettore associato a nome, cognome ed email. Anche l'email di conferma voto invia a quella casella il codice-ricevuta (`vote_id`).

Impatto: Il token è una credenziale al portatore: chi controlla quella casella Gmail può votare al posto di qualunque elettore che non abbia ancora votato e correlare identità e partecipazione. Vengono inoltre concentrati PII e credenziali in un account di terza parte.

Scenario: Compromissione (o uso improprio) della casella di backup → voto fraudolento su larga scala e de-anonimizzazione.

Raccomandazione: Non inviare copie delle credenziali individuali a caselle condivise; se serve tracciare l'invio, registrare solo metadati non sensibili; proteggere l'eventuale casella istituzionale con 2FA e titolarità controllata; valutare token monouso legati alla sessione.

V6 — Segretezza del voto vanificabile per correlazione temporale

Severità	Alta
Probabilità	Media
Stato	Certa
Categoria	OWASP A04:2021 – Insecure Design · CWE-359
Posizione	<code>voting/views.py:237-250</code> · <code>voting/admin.py</code>

Descrizione: Nella stessa transazione atomica vengono creati il record Voter (chiave = token dell'elettore, riconducibile all'identità tramite `local.csv`) e i record Vote della relativa scheda. Voter e Vote risultano quindi scritti con timestamp coincidenti e identificativi auto-incrementali contigui. Entrambi i modelli sono inoltre esposti nell'area `/admin`.

Impatto: Chi disponga di accesso al database (o all'area admin) può ricostruire la corrispondenza tra elettore e scheda mediante correlazione per ordine di inserimento e orario, vanificando la segretezza del voto. La separazione fra `voteKey` ed `emailKey` è, in queste condizioni, solo apparente.

Scenario: Un amministratore o chiunque acceda al file di database correla le righe Voter/Vote della medesima transazione risalendo dall'identità alla preferenza.

Raccomandazione: Disaccoppiare, nel tempo e nella struttura, la registrazione dell'avvenuto voto dalla memorizzazione della scheda (scritture separate, batch con rimescolamento, ritardi randomizzati); rimuovere Vote/Voter dall'admin o limitarne fortemente la visibilità; valutare uno schema di urna crittografica (voto cifrato con scrutinio separato).

V8 — Django 3.2.7 in End-of-Life con CVE note non corrette

Severità	Alta
Probabilità	Media
Stato	Certa
Categoria	OWASP A06:2021 – Vulnerable and Outdated Components
Posizione	<code>requirements.txt:1</code> (e dipendenze correlate)

Descrizione: Il progetto è vincolato a Django 3.2.7 (settembre 2021). Il ramo 3.2 è in End-of-Life dall'aprile 2024 e non riceve più patch di sicurezza. La 3.2.7 è antecedente a correzioni note come CVE-2022-28346/28347 (SQL injection, risolte in 3.2.13) e non incorpora le correzioni successive; le CVE emerse dopo l'EOL (es. CVE-2024-42005, SQLi con CVSS 9.8) non sono state retro-portate. Anche pandas 1.3.3, xlrd 2.0.1 e django-simple-captcha 0.5.14 sono datati.

Impatto: Esecuzione su una base non più mantenuta con vulnerabilità note. Alcune CVE potrebbero non essere direttamente raggiungibili date le funzionalità effettivamente usate (SQLite, uso limitato dell'ORM), ma la superficie resta non presidiata e non conforme alle buone pratiche (e a requisiti quali PCI-DSS 6.3.3 sull'aggiornamento).

Scenario: Sfruttamento di una vulnerabilità nota del framework qualora il percorso di codice risulti raggiungibile in produzione.

Raccomandazione: Aggiornare a una versione supportata (Django 4.2 LTS o 5.x) e aggiornare le altre dipendenze; introdurre una scansione automatica delle dipendenze (es. pip-audit) nel processo di rilascio.

V9 — Cookie di sessione e trasporto privi di direttive di sicurezza

Severità	Alta
Probabilità	Media
Stato	Certa
Categoria	OWASP A05:2021 – Security Misconfiguration · CWE-614/CWE-1004/CWE-319
Posizione	mensa_italia_voting/settings.py (direttive assenti)

Descrizione: Mancano le direttive di sicurezza di sessione/trasporto: SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, SESSION_COOKIE_SAMESITE, SECURE_SSL_REDIRECT, SECURE_HSTS_SECONDS e correlate. SecurityMiddleware è presente ma non configurato.

Impatto: I cookie di sessione dell'area amministrativa possono transitare su HTTP e risultano più esposti a intercettazione (MITM) e a furto di sessione; assenti l'HSTS e il redirect forzato a HTTPS.

Scenario: Intercettazione del cookie di sessione admin su rete non fidata in assenza di TLS forzato.

Raccomandazione: Impostare i cookie come Secure e SameSite=Lax/Strict; forzare HTTPS (SECURE_SSL_REDIRECT); abilitare HSTS; mantenere SESSION_COOKIE_HTTPONLY (default attivo); verificare la terminazione TLS a monte.

V7 — Ricevuta verificabile: rischio di coercizione e voto di scambio

Severità	Media
Probabilità	Media
Stato	Certa
Categoria	OWASP A04:2021 – Insecure Design (receipt-freeness)
Posizione	voting/views.py:267-291 · voting/templates/verify.html

Descrizione: Dopo il voto il sistema restituisce un codice (vote_id) con cui, tramite /verify, si visualizzano le preferenze espresse. Il codice costituisce di fatto una ricevuta che prova il contenuto del voto.

Impatto: Violazione della proprietà di "receipt-freeness": un elettore può dimostrare a terzi come ha votato, abilitando coercizione e voto di scambio. È un rischio di disegno, indipendente da difetti implementativi.

Scenario: Un terzo richiede all'elettore il vote_id per verificarne le scelte, condizionandone il voto.

Raccomandazione: Consentire la verifica dell'avvenuta registrazione senza rivelare il contenuto della scheda, oppure adottare tecniche note di voto verificabile end-to-end con proprietà di receipt-freeness. Scelta da coordinare con il regolamento elettorale.

V10 — Limite ai campi POST disattivato: proliferazione record e DoS

Severità	Media
Probabilità	Media
Stato	Certa
Categoria	OWASP A05:2021 · CWE-770
Posizione	mensa_italia_voting/settings.py:153 · voting/views.py:242

Descrizione: DATA_UPLOAD_MAX_NUMBER_FIELDS = None disattiva la protezione di Django contro l'eccesso di campi in una richiesta (default 1000). In combinazione con la view di voto — che crea un record per ciascun campo POST senza validazione — un singolo elettore può inviare una richiesta con un numero elevatissimo di campi.

Impatto: Denial-of-service applicativo e proliferazione di record nel database (integrità e disponibilità). Amplifica V4.

Scenario: Invio di una singola richiesta POST con decine di migliaia di campi da parte di un avente diritto.

Raccomandazione: Ripristinare un limite ragionevole (default o valore calibrato); validare e limitare il numero di preferenze accettate; introdurre limiti alla dimensione del corpo della richiesta a monte.

V11 — Password amministratore distribuita via email e BCC

Severità	Media
Probabilità	Media
Stato	Certa
Categoria	OWASP A07:2021 – Identification and Authentication Failures · CWE-522
Posizione	voting/management/commands/create_admin.py:27-37

Descrizione: Il comando create_admin genera la password del superuser e la invia via email in chiaro all'indirizzo amministrativo, con ulteriore copia (BCC) verso la casella Gmail di backup.

Impatto: La password dell'utenza con privilegi massimi risiede in almeno due mailbox in chiaro; il furto di una delle caselle comporta la compromissione dell'area amministrativa.

Scenario: Compromissione della casella amministrativa o della casella di backup → accesso admin completo.

Raccomandazione: Non trasmettere password via email; adottare procedure di primo accesso con reset forzato o consegna out-of-band; evitare il BCC verso caselle condivise; imporre la 2FA sull'area amministrativa.

V13 — Helper di servizio file statici di Django in produzione

Severità	Media
Probabilità	Bassa
Stato	Possibile
Categoria	OWASP A05:2021 – Security Misconfiguration
Posizione	voting/views.py:300-313 · urls

Descrizione: Nel codice è ridefinita la funzione helper static()/serve() di Django per servire file. Il servizio statico integrato non è pensato per la produzione (assenza di hardening e caching). Allo stato non risulta collegata a urlpatterns: il rischio è quindi latente/condizionato al suo effettivo utilizzo.

Impatto: Se attivata in produzione, servirebbe i file statici (STATIC_ROOT = BASE_DIR/static) tramite un meccanismo non irrobustito; la protezione anti-traversal di Django generalmente regge, ma la pratica è sconsigliata.

Scenario: Attivazione del pattern serve() in ambiente esposto senza reverse proxy.

Raccomandazione: Servire gli statici tramite web server/reverse proxy (nginx) o WhiteNoise; non usare serve() in produzione; rimuovere il codice se inutilizzato.

V14 — Login amministrativo privo di rate limiting e CAPTCHA

Severità	Media
Probabilità	Bassa
Stato	Possibile
Categoria	OWASP A07:2021 · CWE-307
Posizione	mensa_italia_voting/urls.py (accounts/, admin) · login

Descrizione: Le pagine di autenticazione (area /admin e /accounts) non prevedono limitazione dei tentativi, blocco temporaneo o CAPTCHA. La libreria django-simple-captcha è installata ma non risulta impiegata nel flusso di login né in quello di voto.

Impatto: Espone l'utenza amministrativa a tentativi di forza bruta e credential stuffing. Il token di voto (UUIDv4) è invece robusto alla forza bruta.

Scenario: Attacco automatizzato di indovino password contro /admin/login.

Raccomandazione: Introdurre rate limiting e lockout progressivo sui login (es. django-axes), CAPTCHA dopo N tentativi e 2FA per l'area amministrativa.

V15 — SQLite come database di produzione

Severità	Media
Probabilità	Media

Stato	Possibile
Categoria	OWASP A04:2021 – Insecure Design (affidabilità)
Posizione	mensa_italia_voting/settings.py:84-89

Descrizione: Il database di default è SQLite (file db.sqlite3). SQLite serializza le scritture (lock a livello di file) ed è poco adatto a carichi concorrenti come una finestra di voto con accessi simultanei.

Impatto: Sotto concorrenza sono possibili errori "database is locked" e transazioni fallite (disponibilità e integrità della registrazione). Inoltre l'intero stato (urna e aventi diritto) è un unico file, facilmente esfiltrabile in caso di compromissione dell'host.

Scenario: Picco di voti simultanei allo scadere della finestra → registrazioni fallite; oppure copia del singolo file db.sqlite3.

Raccomandazione: Adottare un RDBMS server (PostgreSQL) per la produzione; se SQLite resta in scenari a bassa concorrenza, cifrare/isolare il file, con backup e controllo degli accessi.

V17 — conf.json e template email modificabili via web

Severità	Media
Probabilità	Bassa
Stato	Possibile
Categoria	OWASP A04/A01:2021
Posizione	voting/views.py:144-171

Descrizione: La funzione /management/edit consente di leggere e sovrascrivere conf.json e i template email (elenco chiuso di tre file). L'accesso è riservato ai superuser, ma la scrittura è un'operazione ad alto impatto e, in assenza di protezione CSRF (V1), risulta ulteriormente esposta.

Impatto: Un attore che ottenga (o forzi via CSRF) una sessione admin può riscrivere i parametri elettorali (candidati, finestra, modalità) o il contenuto delle email inviate ai soci. Funziona da amplificatore di V1.

Scenario: CSRF verso /management/edit su amministratore autenticato → riscrittura di conf.json.

Raccomandazione: Restringere ulteriormente l'operazione (conferma esplicita, audit log, controllo d'integrità dei file), applicare la protezione CSRF, versionare le modifiche a conf.json, applicare il principio del privilegio minimo.

V18 — Gestione errori disabilitata e DEBUG attivabile: rischio info leak

Severità	Media
Probabilità	Bassa
Stato	Possibile
Categoria	OWASP A05:2021 · CWE-209/CWE-215
Posizione	voting/views.py:68-72 · settings.py:27

Descrizione: In `mgmt_upload` il blocco `try/except` di gestione errori è commentato: un'eccezione durante l'elaborazione del file risalirebbe non gestita. `DEBUG` è controllato da variabile d'ambiente (default `False`) ma resta attivabile. Sono inoltre presenti `print()` diagnostici (`BASE_DIR`, `dataframe`) verso lo standard output/log.

Impatto: Con `DEBUG` attivo in produzione, un errore mostrerebbe traceback e impostazioni (inclusi potenzialmente segreti) nella pagina di errore. I `print` possono riversare dati sensibili nei log.

Scenario: Upload malformato con `DEBUG` erroneamente attivo → pagina di errore con traceback e configurazione.

Raccomandazione: Garantire `DEBUG=False` in produzione (verifica in fase di deploy); implementare gestione degli errori e pagine 4xx/5xx personalizzate; rimuovere i `print` diagnostici o sostituirli con logging controllato.

V12 — Token di voto trasmesso come parametro nell'URL

Severità	Bassa
Probabilità	Media
Stato	Certa
Categoria	OWASP A09:2021 · CWE-598
Posizione	<code>voting/views.py:180</code>

Descrizione: Il token individuale di voto è passato come parametro di query (`?key=...`) e letto da `request.GET`. I parametri di query finiscono tipicamente nei log del web server, nella cronologia del browser e negli eventuali proxy.

Impatto: Possibile esposizione del token (credenziale al portatore) in registri e cronologie, con rischio di riuso se i log sono accessibili. Sul fronte `Referer` il rischio è mitigato dalle policy di default dei browser moderni, ma non sui log.

Scenario: Accesso ai log del server o alla cronologia di un dispositivo condiviso → recupero del token e voto per conto dell'elettore (se non ha ancora votato).

Raccomandazione: Trasmettere il token via POST o header; evitarne la comparsa negli URL; ridurre la ritenzione e proteggere i log; considerare token monouso a breve scadenza.

V16 — Invio email bloccante nel contesto della richiesta

Severità	Bassa
Probabilità	Media
Stato	Possibile
Categoria	OWASP A04:2021 · CWE-400
Posizione	<code>voting/views.py:100-118</code>

Descrizione: In `mgmt_send` il "thread" di invio è avviato con `t.run()` (esecuzione sincrona, non `t.start()`) e `do_send` esegue uno `sleep` di 10 secondi per ciascuna email. L'invio avviene quindi interamente nel contesto della richiesta HTTP.

Impatto: La richiesta resta bloccata per un tempo proporzionale al numero di destinatari, con rischio di timeout e di occupazione dei worker e conseguente degrado del servizio. Difetto di affidabilità con potenziale impatto sulla disponibilità.

Scenario: Invio a molte centinaia di destinatari → richiesta in timeout e worker saturi.

Raccomandazione: Delegare l'invio a un task asincrono/coda (es. Celery) o a un management command fuori dal ciclo richiesta; correggere `t.run()` → `t.start()` se si intende usare i thread; gestire ritenti ed errori.

5. Correzioni già presenti nel codice

Nel corso della revisione sono stati rilevati interventi correttivi già applicati, che è corretto riconoscere:

- Doppio voto: la registrazione usa `get_or_create` atomico sulla chiave primaria `emailKey` entro `transaction.atomic()` (`views.py:237-241`), chiudendo la race condition del voto multiplo.
- Backdoor rimosso: eliminato il bypass `"secret" == MENSA_SECRET` che consentiva a richieste anonime di innescare l'invio delle credenziali.
- Controlli lato server sul POST di voto: chiave valida e finestra temporale verificate anche in fase di invio, non solo al rendering della scheda (`views.py:222-229`).
- Transazione atomica sull'intera scheda: una preferenza malformata annulla l'inserimento, evitando un "votante" registrato senza voti.
- Igiene del repository: `.gitignore` esclude `db.sqlite3`, `local.csv`, `*.csv`, `sent.json` e il file delle credenziali admin.
- Integrità delle risorse esterne: i CDN (Bootstrap, Ace) sono caricati con attributi integrity (SRI) e `crossorigin`.

6. Raccomandazioni prioritzate

Immediata (0–7 giorni)

- Revocare e rigenerare le credenziali email e la password amministratore esposte; sostituire la `SECRET_KEY` e spostare tutti i segreti in variabili d'ambiente (V2, V11, V3).
- Riabilitare `CsrfViewMiddleware` (V1).
- Validare lato server le preferenze e ripristinare il limite ai campi POST (V4, V10).
- Eliminare il BCC dei token individuali verso la casella condivisa (V5).

Breve termine (2–4 settimane)

- Aggiornare Django a una versione supportata e le altre dipendenze; introdurre `pip-audit` (V8).
- Applicare l'hardening di cookie e trasporto (Secure, SameSite, HSTS, HTTPS forzato) (V9).
- Introdurre rate limiting e 2FA sull'area amministrativa (V14).
- Rimuovere il token di voto dagli URL, trasmettendolo via POST (V12).

Medio termine (1–3 mesi)

- Rivedere lo schema di urna per garantire la segretezza del voto (V6) e valutare la receipt-freeness (V7).
- Migrare il database a PostgreSQL (V15); rendere asincrono l'invio email (V16).
- Irrobustire l'editor di configurazione con audit e controllo d'integrità (V17); ripristinare la gestione errori e il logging controllato (V18, V13).

7. Considerazioni di competenza e protezione dei dati

Profilo elettorale. I rilievi V4, V6 e V7 incidono direttamente su integrità e segretezza del voto, principi cardine dell'ordinamento elettorale associativo: la loro trattazione è opportuno sia coordinata con il regolamento elettorale e con gli organi competenti.

Protezione dei dati. L'esposizione di dati personali e credenziali (V5, V11 e, in subordine, V12) ha rilevanza ai fini della normativa sulla protezione dei dati personali (GDPR): si consiglia una valutazione da parte del titolare del trattamento, con applicazione dei principi di minimizzazione e di limitazione dell'accesso.

Natura del documento. La presente relazione ha finalità tecnica; non costituisce parere legale. Le valutazioni sono basate esclusivamente sull'analisi statica del codice alla data indicata.

8. Appendici

A. Riferimenti

- OWASP Top 10 (2021) — <https://owasp.org/Top10/>
- Django — stato di supporto (End-of-Life del ramo 3.2: aprile 2024) — <https://endoflife.date/django>
- CVE-2022-28346 / CVE-2022-28347 (SQL injection, corrette in Django 3.2.13) — <https://www.djangoproject.com/weblog/2022/apr/11/security-releases/>
- CVE-2024-42005 (SQL injection, CVSS 9.8; ramo 3.2 non più supportato) — <https://github.com/advisories/GHSA-pv4p-cwwg-4rph>

B. Glossario essenziale

- CSRF (Cross-Site Request Forgery): induzione di una richiesta non voluta da parte di un utente autenticato.
- Credenziale al portatore (bearer): segreto che, da solo, autorizza un'azione (qui, il token di voto).
- Receipt-freeness: proprietà per cui l'elettore non può dimostrare a terzi come ha votato.
- End-of-Life (EOL): fase in cui un software non riceve più aggiornamenti, incluse le patch di sicurezza.

C. Avvertenza

Documento riservato. Contiene informazioni sulle debolezze di un sistema in uso: la diffusione non controllata può agevolarne l'abuso. Le raccomandazioni vanno validate in ambiente di prova prima dell'applicazione in produzione.